

COMP - 8590

Statistical Learning

Assignment 1

Group Member:

Yuhan Xiang	110182721
Xinyi Li	110203076
Qiaoqiao Han.	110187843
Faseeh U Rehman Qureshi	110188603

Through this assignment, we practiced using the **ISLP package in Python** to work through Chapter 2 of *Introduction to Statistical Learning*. In the Lab section, we became familiar with importing datasets, performing basic exploratory data analysis, and creating visualizations. In the conceptual exercise, we gained a deeper understanding of the distinctions between statistical learning and machine learning, as well as the trade-offs between model complexity, interpretability, and predictive accuracy. In the applied exercise (Exercise 8), we implemented linear regression on real data, which strengthened our grasp of the underlying assumptions, model fitting, and interpretation of results. Altogether, these tasks not only improved our comfort with Python tools but also gave us a clearer understanding of the key ideas and methods behind statistical learning.

Below are the screenshots of our completed work for the three questions, and we will also provide the source code files separately.

Question 1

#2.3.1 Getting start

installing and importing the required Python packages (ISLP, pandas, numpy, matplotlib)

Lab 2.3.1 - Requirements

This notebook requires:

- Python 3.x
- ISLP package
- pandas, numpy, matplotlib

[1]
✓ 29
秒

```
# Uncomment the following line if running in Google Colab
!pip install ISLP
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from ISLP import load_data
```



Collecting ISLP

```
Downloading ISLP-0.4.0-py3-none-any.whl.metadata (7.0 kB)
Requirement already satisfied: numpy>=1.7.1 in /usr/local/lib/python3.12/dist-packages (1.26.4)
Requirement already satisfied: scipy>=0.9 in /usr/local/lib/python3.12/dist-packages (1.13.1)
Requirement already satisfied: pandas>=0.20 in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: lxml in /usr/local/lib/python3.12/dist-packages (5.3.0)
Requirement already satisfied: scikit-learn>=1.2 in /usr/local/lib/python3.12/dist-packages (1.5.2)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (1.4.2)
Requirement already satisfied: statsmodels>=0.13 in /usr/local/lib/python3.12/dist-packages (0.14.2)
Collecting lifelines (from ISLP)
Downloading lifelines-0.30.0-py3-none-any.whl.metadata (3.2 kB)
Collecting pygam (from ISLP)
```

#2.3.2 Basic Commands Practise

[4]
✓
0 秒

```
print('fit a model with', 11, 'variables')
```

⇒ fit a model with 11 variables

[5]
✓
0 秒

```
print?
```

[6]
✓
0 秒

```
3 + 5
```

⇒ 8

[7]
✓
0 秒

```
"hello" + " " + "world"
```

⇒ 'hello world'

[8]
✓
0 秒

```
x = [3,4,5]  
x
```

⇒ [3, 4, 5]

[9]
✓
0 秒

```
y = [4, 9, 7]  
x+y
```

⇒ [3, 4, 5, 4, 9, 7]

#2.3.3 Numerical Python :

Using the NumPy package for numerical computations and data manipulation in Python

[11]
✓
0 秒

```
import numpy as np
```

[12]
✓
0 秒

```
x = np.array([3,4,5])  
y = np.array([4,9,7])  
x+y
```

⇒ array([7, 13, 12])

[13]
✓
0 秒

```
x = np.array([[1, 2], [3, 4]])  
x
```

⇒ array([[1, 2],
[3, 4]])

[14]
✓
0 秒

```
x.ndim
```

⇒ 2

[15]
✓
0 秒

```
x.dtype
```

⇒ dtype('int64')

[16]
✓
0 秒

```
np.array([[1, 2], [3.0, 4]]).dtype
```

⇒ dtype('float64')

```
[27]
0 秒
✓ x_reshape.shape, x_reshape.ndim, x_reshape.T

((2, 3),
 2,
 array([[5, 4],
        [2, 5],
        [3, 6]]))

[28]
0 秒
✓ np.sqrt(x)

array([2.23606798, 1.41421356, 1.73205081, 2.         , 2.23606798,
       2.44948974])

[29]
0 秒
✓ x**2

array([25,  4,  9, 16, 25, 36])

[30]
0 秒
✓ x**0.5

array([2.23606798, 1.41421356, 1.73205081, 2.         , 2.23606798,
       2.44948974])

[31]
0 秒
✓ x = np.random.normal(size=50)
x

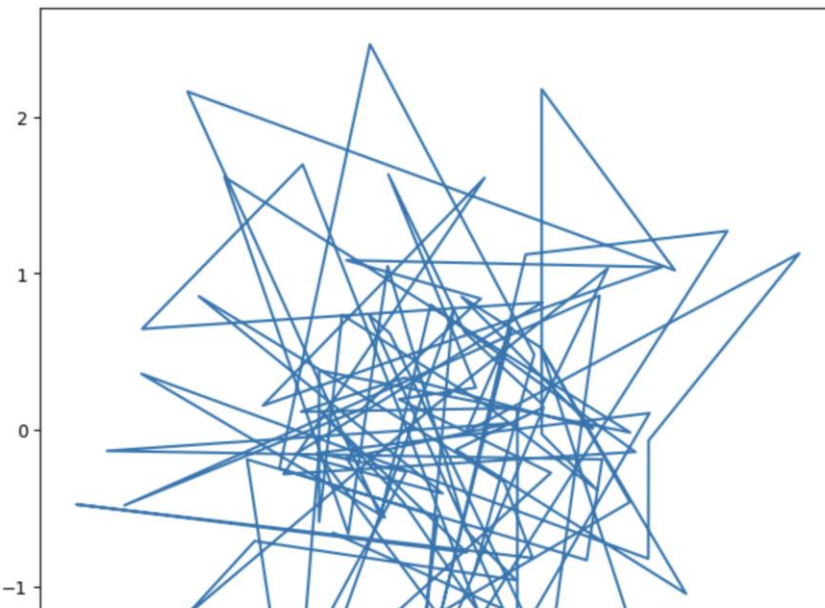
array([-0.06981114,  1.04316233, -1.08964844,  1.57136379, -0.46898273,
       -0.42645823, -0.39149118,  1.51850293, -0.05303694, -0.29982515,
       -1.50164261, -1.18798326,  0.33606588, -1.19744186,  1.94209145,
        0.22850887, -0.50140884, -0.93241324, -0.78319167, -0.96406053,
        2.01259086, -0.20941491,  0.27092217, -0.5432239 , -0.95770563,
        0.14876826, -1.9875942 , -1.34853971,  0.41843375,  1.17910421,
        1.47430464, -0.11435424, -1.19393433,  0.0945833 ,  0.76270286,
        0.14876826, -1.9875942 , -1.34853971,  0.41843375,  1.17910421,
        1.47430464, -0.11435424, -1.19393433,  0.0945833 ,  0.76270286])
```

#2.3.4 Graphics

In this section, we mainly practiced using Matplotlib to create figures and axes with the `subplots()` function and to generate basic plots in Python.

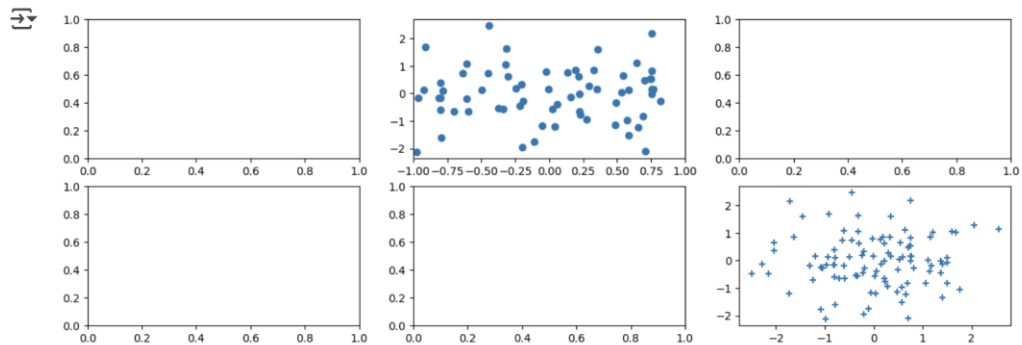
```
[46]
0 秒
✓ from matplotlib.pyplot import subplots
fig, ax = subplots(figsize=(8,8))
x= rng.standard_normal(100)
y= rng.standard_normal(100)
ax.plot(x,y)

[<matplotlib.lines.Line2D at 0x7fec4368f260>]
```



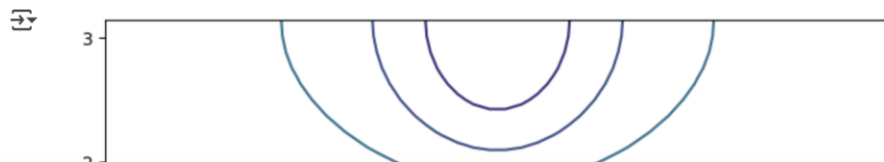
[55]
0
秒

```
axes[0,1].set_xlim([-1,1])  
fig.savefig("Figure_updated.jpg")  
fig
```



[56]
0
秒

```
fig , ax = subplots (figsize =(8, 8))  
x = np.linspace (-np.pi , np.pi , 50)  
y = x  
f = np.multiply .outer(np.cos(y), 1 / (1 + x**2))  
ax.contour(x, y, f);
```



#2.3.5 Sequences and Slice Notation

In this section, we mainly practiced creating sequences with NumPy functions like `linspace` and `arange`, and using slice notation to index and extract elements from sequences in Python.

[60]
0
秒

```
seq1 = np.linspace(0,10,11)  
seq1
```

```
array([ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.])
```

[61]
0
秒

```
seq2 = np.arange(0,10)  
seq2
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

[62]
0
秒

```
"hello world"[3:6]
```

```
'lo '
```

[63]
0
秒

```
"hello world"[slice(3,6)]
```

```
'lo '
```

[64]
0
秒

```
#2.3.6 Indexing Data
```

[65]
0
秒

```
A = np.array(np.arange(16)).reshape((4,4))  
A
```

```
array([[ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11],  
       [12, 13, 14, 15])
```

#2.3.6 Indexing Data

In this section, I mainly practiced indexing and subsetting data from arrays and other data structures in Python.

```
[65] A = np.array(np.arange(16).reshape((4,4)))
      A
      array([[ 0,  1,  2,  3],
             [ 4,  5,  6,  7],
             [ 8,  9, 10, 11],
             [12, 13, 14, 15]])
```

+ 代码 + 文本

```
[66] A[1,2]
      np.int64(6)
```

```
[67] A[[1,3]]
      array([[ 4,  5,  6,  7],
             [12, 13, 14, 15]])
```

```
[68] A[:,[0,2]]
      array([[ 0,  2],
             [ 4,  6],
             [ 8, 10],
             [12, 14]])
```

```
[69] A[[1,3],[0,2]]
      array([ 4, 14])
```

#2.3.7 Loading Data

Screenshots of parts of the exercises

```
[83] Auto = pd.read_csv(dataPath, delim_whitespace=True)
      Auto
```

/tmp/ipython-input-2768897978.py:1: FutureWarning: The 'delim_whitespace' keyword in pd.read_csv is deprecated. Use 'sep='\s+' instead.

```
Auto = pd.read_csv(dataPath, delim_whitespace=True)
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	year	origin	name
0	18.0	8	307.0	130.0	3504.0	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165.0	3693.0	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150.0	3436.0	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150.0	3433.0	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140.0	3449.0	10.5	70	1	ford torino
...	...	...	...	...	...	...	...	...	...
392	27.0	4	140.0	86.00	2790.0	15.6	82	1	ford mustang gl
393	44.0	4	97.0	52.00	2130.0	24.6	82	2	vw pickup
394	32.0	4	135.0	84.00	2295.0	11.6	82	1	dodge rampage

[101]
✓
0 秒

```
Auto_re.iloc [[3 ,4] , [0 ,2 ,3]]
```

	mpg	displacement	horsepower
amc rebel sst	16.0	304.0	150.0
ford torino	17.0	302.0	140.0

[102]
✓
0 秒

```
Auto_re.loc['ford galaxie 500', ['mpg', 'origin']]
```

	mpg	origin
ford galaxie 500	15.0	1
ford galaxie 500	14.0	1
ford galaxie 500	14.0	1

#2.3.8 For Loops

Screenshots of parts of the exercises

[108]
✓
0 秒

```
total=0
for value in [3,2,19]:
    total += value
print('Total is :{0}'.format(total))
```

Total is :24

[109]
✓
0 秒

```
total=0
for value in [3,2,19]:
    for weight in [3,2,1]:
        total += value * weight
print('Total is :{0}'.format(total))
```

Total is :144

[110]
✓
0 秒

```
total=0
for value, weight in zip([3,2,19],[0.2,0.3,0.5]):
    total += weight * value
print('Weighted average is :{0}'.format(total))
```

Weighted average is :10.7

[111]
✓
0 秒

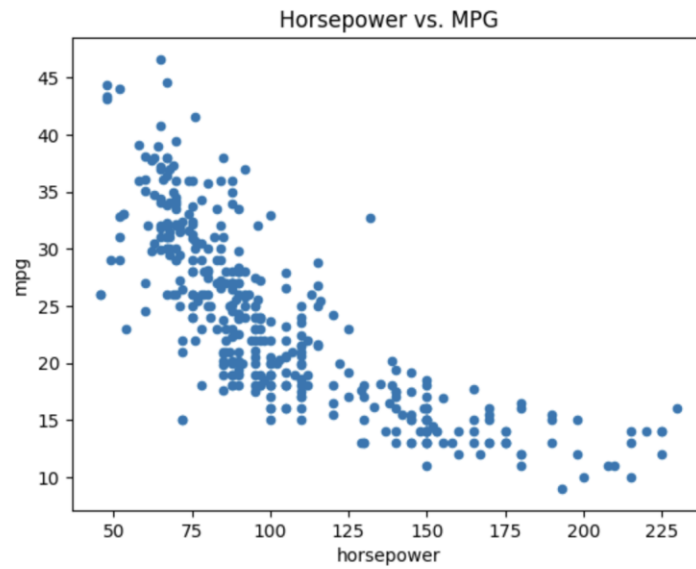
```
rng = np.random. default_rng (1)
A = rng. standard_normal ((127 , 5))
M = rng.choice ([0, np.nan], p=[0.8 ,0.2] , size=A.shape)
A += M
D = pd. DataFrame (A, columns=['food ',
'bar',
'pickle',
'snack',
```

#2.3.9 Additional Graphical and Numerical Summaries

[115]
✓ 0 秒

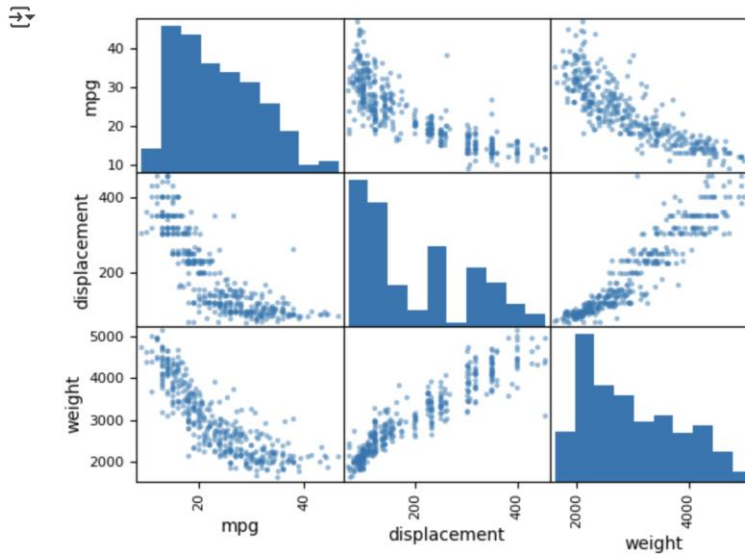
```
ax = Auto.plot.scatter('horsepower', 'mpg');  
ax.set_title('Horsepower vs. MPG')
```

```
Text(0.5, 1.0, 'Horsepower vs. MPG')
```



[123]
✓ 0 秒

```
pd.plotting.scatter_matrix(Auto[['mpg', 'displacement', 'weight']]);
```



Question 2

1. For each of parts (a) through (d), indicate whether we would generally expect the performance of a flexible statistical learning method to be better or worse than an inflexible method. Justify your answer.

(a) The sample size n is extremely large, and the number of predictors p is small.

Since the sample size is huge but the number of predictors is small, a simple or inflexible method (like linear regression) should already work very well. Using a very flexible method here won't improve much and may even bring extra variance.

(b) The number of predictors p is extremely large, and the number of observations n is small.

Here flexible methods would perform worse. With many predictors and few observations, they are very likely to overfit the data. A more restricted or inflexible method is safer since it prevents the model from chasing noise.

(c) The relationship between the predictors and response is highly non-linear.

Flexible methods perform better. They can capture complex nonlinear patterns that inflexible methods (like linear regression) miss. Here, lower bias outweighs the potential increase in variance.

(d) The variance of the error terms, i.e. $\sigma^2 = \text{Var}(\epsilon)$, is extremely high.

When the noise level is very high, flexible methods tend to perform worse. They might fit the noise instead of the real pattern. Inflexible methods are more robust here.

Question 3

Exercise 8

(a)

(a) Use the `pd.read_csv()` function to read the data into Python. Call the loaded data `college`. Make sure that you have the directory set to the correct location for the data

```
college_ds_file_path = '/content/drive/MyDrive/ALL+CSV+FILES+--+2nd+Edition+--+corrected/ALL CSV FILES - 2nd Edition/College.csv'
```

```
college = pd.read_csv(college_ds_file_path)
```

college

	Unnamed: 0	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad	Outstate	Room.Board	Books	Personal	PHD	Terminal	S.F.Ratio	perc.alumni	Expend	Grad.Rate
0	Abilene Christian University	Yes	1660	1232	721	23	52	2885	537	7440	3300	450	2200	70	78	18.1	12	7041	60
1	Adelphi University	Yes	2186	1924	512	16	29	2683	1227	12280	6450	750	1500	29	30	12.2	16	10527	56
2	Adrian College	Yes	1428	1097	336	22	50	1036	99	11250	3750	400	1165	53	66	12.9	30	8735	54
3	Agnes Scott College	Yes	417	349	137	60	89	510	63	12960	5450	450	875	92	97	7.7	37	19016	59
4	Alaska Pacific University	Yes	193	146	55	16	44	249	869	7560	4120	800	1500	76	72	11.9	2	10922	15
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
772	Worcester State College	No	2197	1515	543	4	26	3089	2029	6797	3900	500	1200	60	60	21.0	14	4469	40
773	Xavier University	Yes	1959	1805	695	24	47	2849	1107	11520	4960	600	1250	73	75	13.3	31	9189	83
774	Xavier University of Louisiana	Yes	2097	1915	695	34	61	2793	166	6900	4200	617	781	67	75	14.4	20	8323	49
775	Yale University	Yes	10705	2453	1317	95	99	5217	83	19840	6510	630	2115	96	96	5.8	49	40386	99
776	York College of Pennsylvania	Yes	2989	1855	691	28	63	2988	1726	4990	3560	500	1250	75	75	18.1	28	4509	99

777 rows x 19 columns

(b)

(b) Look at the data used in the notebook by creating and running a new cell with just the code `college` in it. You should notice that the first column is just the name of each university in a column named something like `Unnamed: 0`. We don't really want pandas to treat this as data. However, it may be handy to have these names for later. Try the following commands and similarly look at the resulting data frames:

```
college2 = pd.read_csv('College.csv', index_col=0)
college3 = college.rename({'Unnamed: 0': 'College'},
axis=1)
college3 = college3.set_index('College')
```

This has used the first column in the file as an index for the data frame. This means that pandas has given each row a name corresponding to the appropriate university. Now you should see that the first data column is `Private`. Note that the names of the colleges appear on the left of the table. We also introduced a new python object above: a dictionary, which is specified by dictionary (key, value) pairs. Keep your modified version of the data with the following:

```
college = college3
```

```
college2 = pd.read_csv(college_ds_file_path, index_col=0)
college2
```

	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad	Outstate	Room.Board	Books	Personal	PhD	Terminal	S.F.Ratio	perc.alumni	Expend	Grad.Rate
Abilene Christian University	Yes	1660	1232	721	23	52	2885	537	7440	3300	450	2200	70	78	18.1	12	7041	60
Adelphi University	Yes	2186	1924	512	16	29	2683	1227	12280	6450	750	1500	29	30	12.2	16	10527	56
Adrian College	Yes	1428	1097	336	22	50	1036	99	11250	3750	400	1165	53	66	12.9	30	8735	54
Agnes Scott College	Yes	417	349	137	60	89	510	63	12960	5450	450	875	92	97	7.7	37	19016	59
Alaska Pacific University	Yes	193	146	55	16	44	249	869	7560	4120	800	1500	76	72	11.9	2	10922	15
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
Worcester State College	No	2197	1515	543	4	26	3089	2029	6797	3900	500	1200	60	60	21.0	14	4469	40
Xavier University	Yes	1959	1805	695	24	47	2849	1107	11520	4960	600	1250	73	75	13.3	31	9189	83
Xavier University of Louisiana	Yes	2097	1915	695	34	61	2793	166	6900	4200	617	781	67	75	14.4	20	8323	49
Yale University	Yes	10705	2453	1317	95	99	5217	83	19840	6510	630	2115	96	96	5.8	49	40386	99
York College of Pennsylvania	Yes	2989	1855	691	28	63	2988	1726	4990	3560	500	1250	75	75	18.1	28	4509	99

```
college3 = college.rename({'Unnamed: 0': 'College'}, axis=1)
college3
```

	College	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad	Outstate	Room.Board	Books	Personal	PhD	Terminal	S.F.Ratio	perc.alumni	Expend	Grad.Rate
0	Abilene Christian University	Yes	1660	1232	721	23	52	2885	537	7440	3300	450	2200	70	78	18.1	12	7041	60
1	Adelphi University	Yes	2186	1924	512	16	29	2683	1227	12280	6450	750	1500	29	30	12.2	16	10527	56
2	Adrian College	Yes	1428	1097	336	22	50	1036	99	11250	3750	400	1165	53	66	12.9	30	8735	54
3	Agnes Scott College	Yes	417	349	137	60	89	510	63	12960	5450	450	875	92	97	7.7	37	19016	59
4	Alaska Pacific University	Yes	193	146	55	16	44	249	869	7560	4120	800	1500	76	72	11.9	2	10922	15
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
772	Worcester State College	No	2197	1515	543	4	26	3089	2029	6797	3900	500	1200	60	60	21.0	14	4469	40
773	Xavier University	Yes	1959	1805	695	24	47	2849	1107	11520	4960	600	1250	73	75	13.3	31	9189	83
774	Xavier University of Louisiana	Yes	2097	1915	695	34	61	2793	166	6900	4200	617	781	67	75	14.4	20	8323	49
775	Yale University	Yes	10705	2453	1317	95	99	5217	83	19840	6510	630	2115	96	96	5.8	49	40386	99
776	York College of Pennsylvania	Yes	2989	1855	691	28	63	2988	1726	4990	3560	500	1250	75	75	18.1	28	4509	99

```
college3 = college3.set_index('College')
college3
```

	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad	Outstate	Room.Board	Books	Personal	PhD	Terminal	S.F.Ratio	perc.alumni	Expend	Grad.Rate
College																		
Abilene Christian University	Yes	1660	1232	721	23	52	2885	537	7440	3300	450	2200	70	78	18.1	12	7041	60
Adelphi University	Yes	2186	1924	512	16	29	2683	1227	12280	6450	750	1500	29	30	12.2	16	10527	56
Adrian College	Yes	1428	1097	336	22	50	1036	99	11250	3750	400	1165	53	66	12.9	30	8735	54
Agnes Scott College	Yes	417	349	137	60	89	510	63	12960	5450	450	875	92	97	7.7	37	19016	59
Alaska Pacific University	Yes	193	146	55	16	44	249	869	7560	4120	800	1500	76	72	11.9	2	10922	15
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
Worcester State College	No	2197	1515	543	4	26	3089	2029	6797	3900	500	1200	60	60	21.0	14	4469	40
Xavier University	Yes	1959	1805	695	24	47	2849	1107	11520	4960	600	1250	73	75	13.3	31	9189	83
Xavier University of Louisiana	Yes	2097	1915	695	34	61	2793	166	6900	4200	617	781	67	75	14.4	20	8323	49
Yale University	Yes	10705	2453	1317	95	99	5217	83	19840	6510	630	2115	96	96	5.8	49	40386	99
York College of Pennsylvania	Yes	2989	1855	691	28	63	2988	1726	4990	3560	500	1250	75	75	18.1	28	4509	99

(c)

(c) Use the describe() method of to produce a numerical summary of the variables in the data set.

college.describe()												
	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad	Outstate	Room.Board	Books	Personal	
count	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000
mean	3001.638353	2018.804376	779.972973	27.558559	55.796654	3699.907336	855.298584	10440.669241	4357.526384	549.380952	1340.642214	
std	3870.201484	2451.113971	929.176190	17.640364	19.804778	4850.420531	1522.431887	4023.016484	1096.696416	165.105360	677.071454	
min	81.000000	72.000000	35.000000	1.000000	9.000000	139.000000	1.000000	2340.000000	1780.000000	96.000000	250.000000	
25%	776.000000	604.000000	242.000000	15.000000	41.000000	992.000000	95.000000	7320.000000	3597.000000	470.000000	850.000000	
50%	1558.000000	1110.000000	434.000000	23.000000	54.000000	1707.000000	353.000000	9990.000000	4200.000000	500.000000	1200.000000	
75%	3624.000000	2424.000000	902.000000	35.000000	69.000000	4005.000000	967.000000	12925.000000	5050.000000	600.000000	1700.000000	
max	48094.000000	26330.000000	6392.000000	96.000000	100.000000	31643.000000	21836.000000	21700.000000	8124.000000	2340.000000	6800.000000	

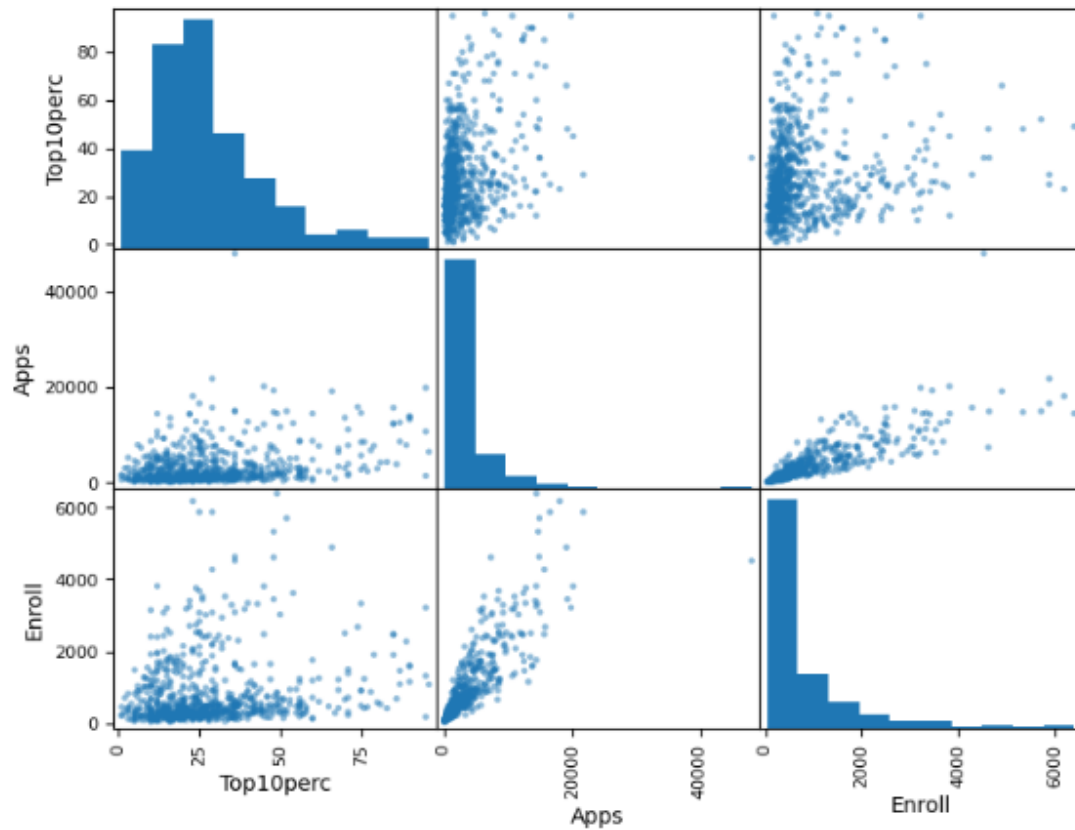
college.describe()																	
	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad	Outstate	Room.Board	Books	Personal	PhD	Terminal	S.F.Ratio	perc.alumni	Expend	Grad.Rate	
777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000
2018.804376	779.972973	27.558559	55.796654	3699.907336	855.298584	10440.669241	4357.526384	549.380952	1340.642214	72.660232	79.702703	14.089704	22.743887	9660.171171	65.46332		
2451.113971	929.176190	17.640364	19.804778	4850.420531	1522.431887	4023.016484	1096.696416	165.105360	677.071454	16.328155	14.722359	3.958349	12.391801	5221.768440	17.17771		
72.000000	35.000000	1.000000	9.000000	139.000000	1.000000	2340.000000	1780.000000	96.000000	250.000000	8.000000	24.000000	2.500000	0.000000	3186.000000	10.00000		
604.000000	242.000000	15.000000	41.000000	992.000000	95.000000	7320.000000	3597.000000	470.000000	850.000000	62.000000	71.000000	11.500000	13.000000	6751.000000	53.00000		
1110.000000	434.000000	23.000000	54.000000	1707.000000	353.000000	9990.000000	4200.000000	500.000000	1200.000000	75.000000	82.000000	13.600000	21.000000	8377.000000	65.00000		
2424.000000	902.000000	35.000000	69.000000	4005.000000	967.000000	12925.000000	5050.000000	600.000000	1700.000000	85.000000	92.000000	16.500000	31.000000	10830.000000	78.00000		
16330.000000	6392.000000	96.000000	100.000000	31643.000000	21836.000000	21700.000000	8124.000000	2340.000000	6800.000000	103.000000	100.000000	39.800000	64.000000	56233.000000	118.00000		

(d)

(d) Use the pd.plotting.scatter_matrix() function to produce a scatterplot matrix of the first columns [Top10perc, Apps, Enroll]. Recall that you can reference a list C of columns of a data frame A using A[C].

```
cols = ["Top10perc", "Apps", "Enroll"]
pd.plotting.scatter_matrix(college[cols], figsize=(8, 6))
```

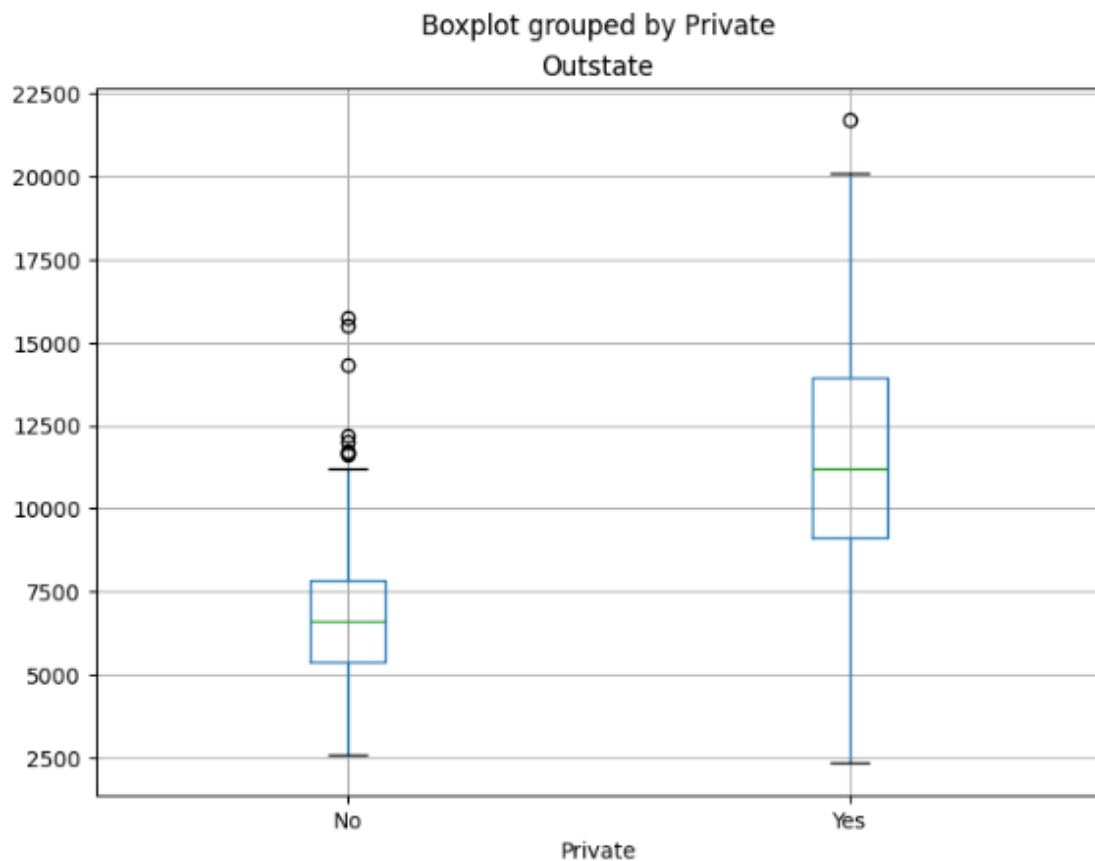
```
array([[<Axes: xlabel='Top10perc', ylabel='Top10perc'>,
       <Axes: xlabel='Apps', ylabel='Top10perc'>,
       <Axes: xlabel='Enroll', ylabel='Top10perc'>],
      [<Axes: xlabel='Top10perc', ylabel='Apps'>,
       <Axes: xlabel='Apps', ylabel='Apps'>,
       <Axes: xlabel='Enroll', ylabel='Apps'>],
      [<Axes: xlabel='Top10perc', ylabel='Enroll'>,
       <Axes: xlabel='Apps', ylabel='Enroll'>,
       <Axes: xlabel='Enroll', ylabel='Enroll'>]], dtype=object)
```



(e)

(e) Use the `boxplot()` method of `college` to produce side-by-side boxplots of `Outstate` versus `Private`.

```
college.boxplot(column="Outstate", by="Private", figsize=(8,6))
```



(f)

(f) Create a new qualitative variable, called Elite, by binning the Top10perc variable into two groups based on whether or not the proportion of students coming from the top 10% of their high school classes exceeds 50%.

```
college['Elite'] = pd.cut(college['Top10perc'],
[0,0.5,1],
labels=['No', 'Yes'])
```

Use the value_counts() method of college['Elite'] to see how many elite universities there are. Finally, use the boxplot() method again to produce side-by-side boxplots of Outstate versus Elite.

```
college['Elite'] = pd.cut(college['Top10perc'], [0,50,100], labels=['No', 'Yes'])
college
```

	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad	Outstate	Room.Board	Books	Personal	PhD	Terminal	S.F.Ratio	perc.alumni	Expend	Grad.Rate	Elite
College																			
Abilene Christian University	Yes	1660	1232	721	23	52	2885	537	7440	3300	450	2200	70	78	18.1	12	7041	60	No
Adelphi University	Yes	2186	1924	512	16	29	2683	1227	12280	6450	750	1500	29	30	12.2	16	10527	56	No
Adrian College	Yes	1428	1097	336	22	50	1036	99	11250	3750	400	1165	53	66	12.9	30	8735	54	No
Agnes Scott College	Yes	417	349	137	60	89	510	63	12960	5450	450	875	92	97	7.7	37	19016	59	Yes
Alaska Pacific University	Yes	193	146	55	16	44	249	869	7560	4120	800	1500	76	72	11.9	2	10922	15	No
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
Worcester State College	No	2197	1515	543	4	26	3089	2029	6797	3900	500	1200	60	60	21.0	14	4469	40	No
Xavier University	Yes	1959	1805	695	24	47	2849	1107	11520	4960	600	1250	73	75	13.3	31	9189	83	No
Xavier University of Louisiana	Yes	2097	1915	695	34	61	2793	166	6900	4200	617	781	67	75	14.4	20	8323	49	No
Yale University	Yes	10705	2453	1317	95	99	5217	83	19840	6510	630	2115	96	96	5.8	49	40386	99	Yes
York College of Pennsylvania	Yes	2989	1855	691	28	63	2988	1726	4990	3560	500	1250	75	75	18.1	28	4509	99	No

```
college['Elite'].value_counts()
```



count

Elite

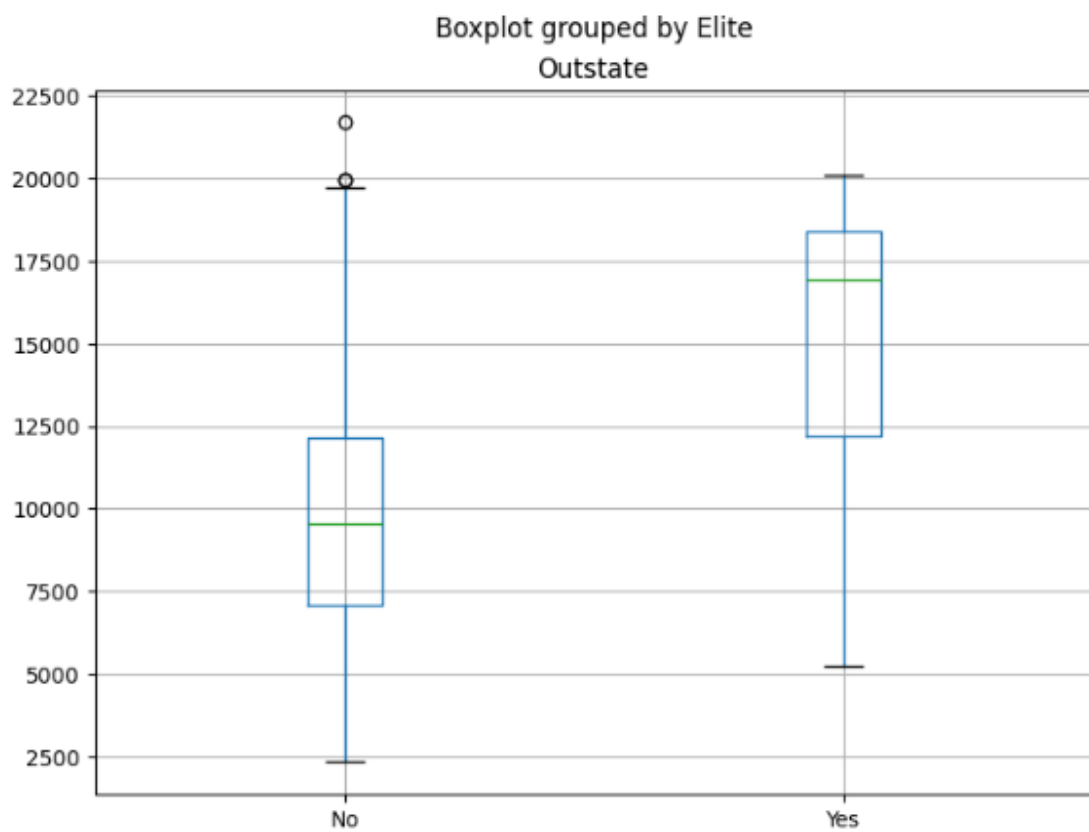
No	699
----	-----

Yes	78
-----	----

dtype: int64

```
college.boxplot(column="Outstate", by="Elite", figsize=(8,6))
```

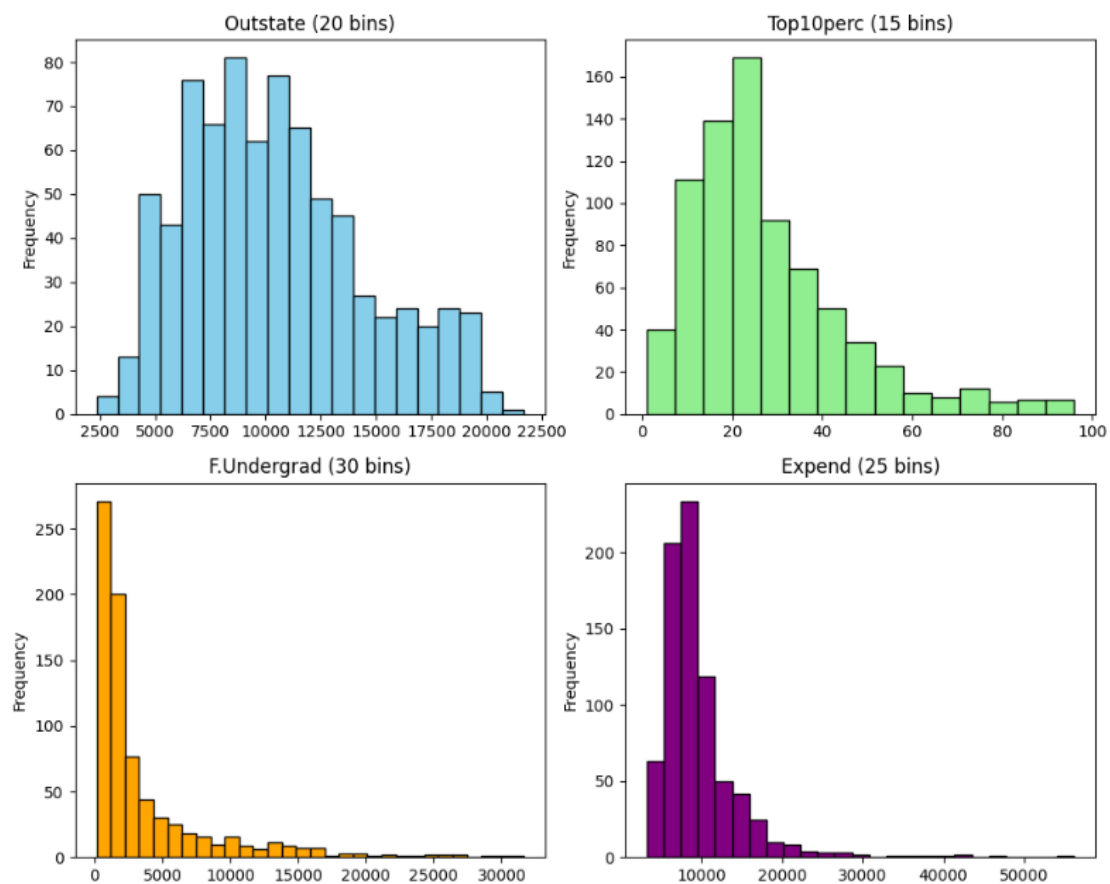
<Axes: title={'center': 'Outstate'}, xlabel='Elite'>



(g)

(g) Use the `plot.hist()` method of `college` to produce some histograms with differing numbers of bins for a few of the quantitative variables. The command `plt.subplots(2, 2)` may be useful: it will divide the plot window into four regions so that four plots can be made simultaneously. By changing the arguments you can divide the screen up in other combinations.

```
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
college['Outstate'].plot.hist(bins=20, ax=axes[0,0], color="skyblue", edgecolor="black")
axes[0,0].set_title("Outstate (20 bins)")
college['Top10perc'].plot.hist(bins=15, ax=axes[0,1], color="lightgreen", edgecolor="black")
axes[0,1].set_title("Top10perc (15 bins)")
college['F.Undergrad'].plot.hist(bins=30, ax=axes[1,0], color="orange", edgecolor="black")
axes[1,0].set_title("F.Undergrad (30 bins)")
college['Expend'].plot.hist(bins=25, ax=axes[1,1], color="purple", edgecolor="black")
axes[1,1].set_title("Expend (25 bins)")
plt.tight_layout()
plt.show()
```



(h)

(h) Continue exploring the data, and provide a brief summary of what you discover.

The data reveal systematic differences between private vs. public and elite vs. non-elite colleges, with elite and private schools generally charging higher tuition and spending more per student. Most colleges are relatively small and admit a modest percentage of students from the top 10% of their high school classes. The dataset shows strong skewness in enrollment and expenditure variables, reflecting the diversity between small liberal arts colleges and large research universities.